

## # Avaliação do Front-end Django - Integração MQTT

**\*\*Data:\*\*** 19 de maio de 2026

---

### ## Resumo Executivo

O front-end Django foi implementado com **\*\*integração completa com MQTT\*\***, possuindo capacidade de **\*\*receber e enviar dados\*\*** do Node-RED. O sistema está bem estruturado, segue boas práticas e apresent

---

### ## Pontos Fortes

#### 1. **\*\*Arquitetura de Comunicação MQTT Completa\*\***

#### 2. **\*\*Recepção de Dados MQTT (Subscriber)\*\***

| Tópico | Descrição | Processamento | Status |

**\*\*Detalhes da implementação (`state.py`)\*\*:**

elif topic.endswith("status"):

elif topic.endswith("atuador/estado"):

#### 3. **\*\*Envio de Comandos MQTT (Publisher)\*\***

| Comando | Tópico | Uso |

**\*\*Implementação (`mqtt.py` - publish\_command)\*\*:**

#### 4. **\*\*APIs RESTful Bem Definidas\*\***

##### GET `/api/state/`

##### POST `/api/command/`

// Response

#### 5. **\*\*Frontend Reativo (JavaScript)\*\***

#### 6. **\*\*Gerenciamento de Estado Thread-Safe\*\***

#### 7. **\*\*Configuração Flexível por Variáveis de Ambiente\*\***

```python

---

### ## Pontos de Atenção

#### 1. **\*\*Validação de Temperatura com Threshold Fixo\*\***

**\*\*Problema\*\*:** Threshold hardcoded em 8°C, não configurável

#### 2. **\*\*Falta de Validação de Autorização na API\*\***

**\*\*Problema\*\*:** Qualquer cliente pode enviar comandos

#### 3. **\*\*Tratamento de Erros em JSON Decode\*\***

**\*\*Problema\*\*:** Corpo vazio retorna `{}` válido, não erro

#### 4. **\*\*Sem Persistência de Dados\*\***

**\*\*Impacto\*\*:** Médio

#### 5. **\*\*Payload Vazio Causa Erro de Conversão\*\***

**\*\*Problema\*\*:** Não valida se `payload` é string válida

#### 6. **\*\*Sem Timeout em Cliente MQTT\*\***

**\*\*Problema\*\*:** Se broker não responder, thread pode travar

---

### ## Fluxo de Comunicação MQTT

...

...

---

### ## Testes Realizados Analiticamente

#### Teste 1: Recepção de Temperatura

**\*\*Resultado esperado\*\*:**

**\*\*Verificação no código\*\*:** Implementado em `ingest\_message`

---

#### Teste 2: Envio de Comando

**\*\*Resultado esperado\*\*:**

**\*\*Verificação no código\*\*:** Implementado em `publish\_command`

---

```
### Teste 3: Múltiplos Brokers
**Resultado esperado**:
**Verificação no código**: Implementado em `_parse_brokers()` e `_worker()`
---
## Estatísticas de Implementação
| Métrica | Valor |
---
## Recomendações de Melhoria
### Prioridade Alta
### Prioridade Média
### Prioridade Baixa
---
## Configuração Necessária
### Arquivo `.env` (Exemplo)
# MQTT
---
## Conclusão
### Resumo Final
**Front-end Django está FUNCIONAL e INTEGRADO com MQTT**
- **Recepção de dados**: Totalmente implementada
### Status de Produção
**Pronto para produção com pequenas melhorias**:
---
**Avaliação geral: 8.5/10**
Sistema bem estruturado, falta apenas melhorias de segurança e persistência para estar 100% pronto para produção.
```